

TaxiBy — Enterprise Mobility Platform

Complete Project Guide, Architectural Trade-offs, and Word-for-Word Spoken Interview Scripts

Candidate: Divesh S **Role:** Software Engineer / Full-Stack & Backend

Primary Stack: NestJS, Next.js 16, PostgreSQL, Redis, RabbitMQ, WebSockets **Location:** Coimbatore, India

1. Project Overview & High-Level Architecture

TaxiBy is a full-stack, enterprise-grade taxi booking and fleet mobility platform built for the Indian market (comparable to Uber or Ola). It handles the complete lifecycle: **ride search** → **dynamic pricing** → **automated dispatch** → **live GPS tracking** → **ride completion** → **digital invoicing** → **driver wallet settlement** → **instant bank payouts**.

Backend Architecture (3 Microservices)

- `apps/core` (Port 4000): Primary business logic, bookings, dynamic pricing engine, wallet ledger, CRM, and Cashfree payouts.
- `apps/tracking` (Port 4002): Dedicated real-time service streaming driver GPS telemetry via WebSockets and Redis in-memory sets.
- `apps/notifications` (Port 4001): Standalone worker consuming 6 RabbitMQ queues for push notifications, SMS, WhatsApp, and SOS alerts.

Frontend Architecture (3 Next.js Portals)

- `taxiby_website` (Port 3000): Public customer booking portal with SEO route pages, Google Places Autocomplete, and Zustand state.
- `taxiby_admin` (Port 3001): Super-admin console for fleet operations, rate cards, analytics, and driver document verification.
- `taxiby_employee` (Port 3002): Operational CRM for call center agents, Exotel IVR routing, and lead conversions.

2. How to Pitch This Project in an Interview

When an interviewer asks you: "Can you walk me through TaxiBy?" or "Tell me about your work at Webnox Technologies."

60-SECOND ELEVATOR PITCH (SAY THIS WORD-FOR-WORD)

"TaxiBy is an enterprise-grade ride-hailing and fleet mobility platform similar to Uber or Ola, built for the Indian market. The backend is architected as a high-concurrency NestJS microservices system comprising three core services: Core for business logic and booking lifecycles, Tracking for real-time driver GPS telemetry over WebSockets, and Notifications for asynchronous multi-channel alerting via RabbitMQ."

"On this project, my primary focus was the core distributed backend: I engineered the real-time driver dispatch and tracking engine using Uber's H3 hexagonal spatial indexing and Redis, which brought spatial query latency down to under 2 milliseconds. I also designed an offline pricing engine combined with a backend Google Maps Directions proxy that cut our external Maps API costs by 70 to 80 percent, and built an immutable double-entry wallet ledger in PostgreSQL to ensure zero race conditions during driver earnings and instant Cashfree payouts."

20-SECOND SHORT PITCH (FOR QUICK SCREENING CALLS)

"I architected the distributed backend for TaxiBy, an enterprise ride-hailing platform using NestJS microservices, Redis, PostgreSQL, and WebSockets. My main contributions included sub-second driver dispatch using Uber's H3 spatial index, cutting Google Maps API costs by 75% via an intelligent Redis proxy, and building an immutable double-entry financial ledger for driver earnings and automated Cashfree payouts."

3. The 4 Golden Pillars (Deep Dive & Spoken Scripts)

Interviewers will test your depth on these 4 architectural achievements. Master the problem, solution, and spoken explanation for each.

🚩 Pillar 1: Sub-Second Driver Dispatch & Real-Time Tracking (Uber H3 + Redis)

The Problem: If hundreds of active drivers stream GPS updates every 3–5 seconds directly to PostgreSQL, the database gets overwhelmed with write IOPS, table bloat, and connection locks. Traditional SQL proximity queries (`WHERE ST_DWithin(...)`) take 100ms–500ms and choke the database.

The Solution:

- Active drivers maintain persistent **WebSockets (Socket.IO)** to the `tracking` microservice.
- Coordinates are converted into 64-bit integer **Uber H3 Hexagonal Indexes (Resolution 7)** and saved into in-memory **Redis Sets** (`h3:<index>:drivers`).
- When a ride is requested, nearest drivers are queried directly in Redis (`$O(1)` lookup taking <2ms), keeping 100% of telemetry write traffic off PostgreSQL.
- Location snapshots are batch-persisted to PostgreSQL only once every 30 seconds for audit history.

🧑 HOW TO EXPLAIN PILLAR 1 IN AN INTERVIEW:

"When drivers are moving, writing GPS pings every 3 seconds to PostgreSQL causes massive write contention and table bloat. To solve this, we decoupled tracking into a dedicated NestJS microservice. Drivers stream coordinates over WebSockets, and we index them in memory using Uber's H3 hexagonal spatial system at Resolution 7 directly in Redis Sets. When the core booking service needs to dispatch a ride, it queries Redis for nearby drivers in that H3 cell and adjacent rings in under 2 milliseconds. PostgreSQL is only updated in batches every 30 seconds for historical snapshots."

🚩 Pillar 2: Google Maps API Cost Optimization (70–80% Bill Reduction)

The Problem: When customers opened the app or viewed route quotes, client apps hit the Google Directions and Distance Matrix APIs directly. Frequent identical routes (e.g. Railway Station to Airport) were queried repeatedly, driving monthly bills to ₹30,000–₹40,000.

The Solution:

- Routed all route requests through a **centralized NestJS backend proxy**.
- Coordinates are rounded to **3 decimal places (~110 meters)** to create an in-memory Redis cache key: `dir:cache:<lat1,lng1>:<lat2,lng2>` with a multi-day TTL.
- For pricing, built an **offline pure-TypeScript pricing engine** that calculates base fare, distance slabs, waiting time, and surge without calling external APIs once distance is determined.
- Reduced external Google Maps API requests by 70–80%, saving ₹25k–₹35k/month.

🧑 HOW TO EXPLAIN PILLAR 2 IN AN INTERVIEW:

"Every upfront ride search was triggering direct calls to Google Directions API, causing high latency and monthly API costs of over ₹35,000. I addressed this by implementing a backend Directions proxy in NestJS that rounds pickup and drop-off coordinates to 3 decimal places—roughly a 100-meter radius—and caches the polyline and distance in Redis. If another rider searches for a route within that 100-meter cluster, we

return the cached route instantly in under 10ms. A 100-meter variance causes a negligible fare difference of a couple of rupees, but it cut our Google Maps bill by 75%."

✦ Pillar 3: Concurrency-Safe Double-Entry Wallet Ledger & Cashfree Payouts

The Problem: In financial platforms, simply updating a `balance` column via `UPDATE users SET balance = balance + x` leads to race conditions, double-spending, and no historical audit trail if disputes occur.

The Solution:

- Implemented a **double-entry ledger pattern**: no code ever mutates wallet balance directly. Every monetary movement (ride earning, commission, payout) inserts an immutable row into `WalletLedger`.
- Wrapped inside **Prisma interactive transactions (`$transaction`)** with row-level locks to prevent concurrent checkout and withdrawal collisions.
- Enforced a database unique constraint on `[driverId, referenceId, transactionType]` for strict **idempotency**—duplicate webhook retries are rejected safely.
- Integrated **Cashfree Payouts API** (IMPS/UPI) with webhook signature verification, automating end-of-day driver settlement.

🧑 HOW TO EXPLAIN PILLAR 3 IN AN INTERVIEW:

"We treated driver money like a banking system. Rather than treating wallet balance as a mutable counter, every financial event creates an append-only row in an immutable `WalletLedger` table. All balance updates and ledger insertions run inside Prisma interactive transactions with row-level locking. We enforced a unique constraint on `driverId`, `referenceId`, and `transactionType` to guarantee idempotency, ensuring that third-party payment retries from Razorpay or Cashfree never credit a driver twice."

✦ Pillar 4: Asynchronous Event-Driven Notifications (RabbitMQ)

The Problem: Calling external third-party communication APIs (WhatsApp, SMS, Firebase Push) synchronously inside the booking flow adds 500ms–2000ms latency. If an external telecom gateway times out, the user's booking request fails.

The Solution:

- The Core API publishes notification events to **RabbitMQ topic exchanges** in <2ms and returns immediately to the client.
- The standalone `notifications` microservice consumes messages from 6 durable queues: `push_notification_queue`, `whatsapp_queue`, `sms_queue`, `web_push_queue`, `sos_queue`, and `driver_reawake_queue`.
- Isolates external telecom failures and guarantees at-least-once delivery with retry handling.

🧑 HOW TO EXPLAIN PILLAR 4 IN AN INTERVIEW:

"Third-party messaging APIs like WhatsApp or SMS gateways have variable latency and occasional rate limits. If we sent notifications synchronously in the HTTP request cycle, a slow SMS provider would delay the ride booking response. By publishing notification events to RabbitMQ durable queues, the booking API responds in under 50 milliseconds, while the dedicated notifications microservice handles delivery to FCM, Interakt WhatsApp, and Fast2SMS asynchronously with retries."

4. Complete Booking Lifecycle (From Search to Payout)

When an interviewer says: "Walk me through how a ride progresses through your system from start to finish."

Stage	State Transition	What Happens Technically
1. Search & Fare	DRAFT	Customer inputs pickup & drop. Backend proxy checks Redis for cached route. Pricing engine calculates base fare + km slabs + time + surge multiplier, returning an immutable fare quote.
2. Confirmation & Dispatch	CONFIRMED → SEARCHING	Customer clicks "Book Now". Booking row is inserted. Core service calls tracking service to find nearest online drivers in Redis H3 cells. Dispatches ride invite via WebSockets.
3. Driver Acceptance	ASSIGNED → ARRIVED	First driver to accept locks the ride via an atomic status check. Rider receives driver details. Driver arrives at pickup; rider verifies 4-digit OTP.
4. Trip in Progress	ONGOING	Driver starts trip. Telemetry streams over WebSockets; rider app tracks live position on map. Distance is calculated along the route.
5. Completion & Settlement	COMPLETED	Driver ends ride. Final fare is finalized. Core service runs Prisma <code>\$transaction</code> : credits driver wallet, records platform commission in <code>WalletLedger</code> , generates invoice, and pushes receipt event to RabbitMQ.
6. Driver Payout	PAYOUT_REQUESTED → PAID	Driver requests payout or automated EOD batch runs. Cashfree Payouts API executes IMPS/UPI bank transfer; webhook verifies settlement and updates ledger.

 **SPOKEN WALKTHROUGH SCRIPT (PRACTICE SAYING THIS):**

"When a rider enters a route, the pricing engine calculates an immutable fare quote using cached distance from Redis and dynamic surge rules. Upon confirmation, the booking enters `SEARCHING` state, and the tracking service queries Redis H3 indexes to find the nearest available drivers and dispatch invites via WebSockets.

Once a driver accepts and verifies the rider's OTP at pickup, the trip transitions to `ONGOING` with live WebSocket tracking. At drop-off, the trip completes, and an atomic database transaction updates the immutable wallet ledger—crediting the driver's earnings and deducting platform commission. Finally, a completion event is published to RabbitMQ to dispatch receipts via WhatsApp and Push, and the driver's earnings become available for automated Cashfree bank transfer."

5. Key Architectural Trade-offs ("Why X Instead of Y")

Senior interviewers love trade-offs. Here is exactly why you made each architectural choice:

1. Why NestJS instead of raw Express?

Spoken: "Express is completely unopinionated. In a system with 62 modules and hundreds of endpoints, Express quickly becomes difficult to maintain. NestJS gave us modular architecture, dependency injection, and decorators, making it easy to keep domain boundaries clean."

2. Why Redis + Uber H3 instead of PostGIS?

Spoken: "PostGIS is great for complex geographic boundaries, but writing high-frequency GPS coordinates from moving drivers creates massive write IOPS and CPU contention on PostgreSQL. Redis operates in memory with sub-millisecond lookups, offloading all ephemeral pings from the database."

3. Why RabbitMQ instead of Kafka?

Spoken: "Kafka is built for massive event streaming and data replays with millions of events per second. For our use case—transactional notifications and background task dispatching—RabbitMQ was much simpler to operate, providing durable queues, acknowledgments, and fine-grained routing without cluster overhead."

4. Why PostgreSQL instead of MongoDB?

Spoken: "Ride-hailing is fundamentally relational and financial. Bookings link to customers, drivers, vehicles, tariffs, and wallet ledgers. PostgreSQL provides ironclad ACID transactions with row-level locking, ensuring financial accuracy where document stores fall short."

5. Why Razorpay + Cashfree dual payment?

Spoken: "Razorpay provides the highest checkout conversion rates and UPI success rates for Indian customer collections, while Cashfree specializes in high-volume, instant automated bank payouts (IMPS/NEFT) for drivers with lower fees."

6. Why separate Next.js apps per portal?

Spoken: "The customer website is public-facing with heavy SEO and SSR requirements, while the admin and employee portals are private operational SPAs behind authentication. Separate apps allow independent deployments and isolated session cookies."

6. Master Interview Questions & Spoken Answers

Q1: "How do you handle race conditions when two drivers accept the same ride simultaneously?"

SPOKEN ANSWER:

"We handle this using atomic state checks at the database level. When a driver clicks 'Accept', our booking service executes an update query with a conditional where clause: `UPDATE bookings SET status = 'ASSIGNED', driverId = ? WHERE id = ? AND status = 'SEARCHING'`. Since PostgreSQL handles row locking during updates, only the first driver's transaction successfully updates the row count to 1. The second driver receives a row count of 0, and the system immediately returns a friendly message stating that the ride was already assigned to another driver."

Q2: "How does your real-time tracking scale if you have multiple server instances?"

SPOKEN ANSWER:

"Our tracking microservice uses Socket.IO with the `@socket.io/redis-adapter`. This means multiple instances of the tracking gateway can run in parallel behind a load balancer. When a driver sends a location update to Server A, the Redis Pub/Sub adapter propagates that event across all instances, so a rider connected to Server B still receives the live location update seamlessly."

Q3: "How do you prevent a driver from withdrawing their wallet balance twice?"

SPOKEN ANSWER:

"When a driver initiates a withdrawal, we don't immediately call the payment gateway. First, within a Prisma interactive transaction, we check the available balance and immediately lock that amount by inserting a `PAYOUT_LOCKED` ledger entry, deducting it from the available balance. Only after the funds are successfully locked do we call the Cashfree Payouts API. If the gateway transfer succeeds, the transaction is marked `PAID`; if it fails, a compensating `PAYOUT_UNLOCKED` ledger entry restores the funds."

Q4: "What happens if the Google Maps API fails or times out?"

SPOKEN ANSWER:

"We designed the backend proxy with fallback resilience. If Google Maps fails or hits rate limits, the system first checks our Redis cache for an existing nearby route. If no cached route exists, we have an internal Haversine geodesic calculation that computes straight-line distance multiplied by a road-network tortuosity factor (~1.25x for city traffic) to provide an immediate estimated fare so the booking flow is never blocked."

Q5: "How does your Exotel IVR and number masking feature work?"

SPOKEN ANSWER:

"For customer and driver safety, we never expose personal phone numbers. When a rider clicks 'Call Driver', our backend requests Exotel to bridge the call using a virtual proxy number. Exotel hits our webhook to verify that an active trip exists between the two users before connecting them. All call duration and recording metadata is captured via webhooks for dispute resolution."

7. Defending System Weaknesses (Demonstrating Senior Maturity)

Interviewers respect candidates who can honestly critique their own systems and explain the path to production hardening.

Weakness 1: "Did you have authentication on your WebSocket connections?"

HOW TO DEFEND:

"In the early rollout of the tracking service, our primary focus was validating low-latency connection stability and tuning the Redis H3 index throughput. To facilitate rapid mobile client testing and avoid socket reconnect loops, we temporarily deferred strict handshake authentication at the socket layer. The immediate production hardening step we have planned is implementing a NestJS `WsGuard` that validates the JWT in the initial connection handshake headers and rejects unauthorized sockets."

Weakness 2: "How would you scale the database to 10x your current load?"

HOW TO DEFEND:

"Currently, all services connect to a primary PostgreSQL instance. At 10x scale, the first step is adding read replicas with query splitting—routing read-heavy dashboard and analytics queries away from the primary. Second, we would add PgBouncer for database connection pooling. Third, we would migrate location snapshots from relational tables to a time-series store like TimescaleDB or Redis Streams."

Weakness 3: "What would you do differently if you started this project today?"

HOW TO DEFEND:

"Two things: First, I would set up a monorepo build tool like Turborepo from day one to share common UI components and API types across our three Next.js portals instead of maintaining them separately. Second, I would write unit tests for the pricing engine and wallet ledger upfront, because financial calculation regressions are the highest-risk issues in mobility platforms."

8. Interview Cheatsheet: What to Say vs. What NOT to Say

Keep your interview delivery aligned with the skills on your resume and avoid bringing up technologies you removed.

Topic	✔ What to Say (Confident & Prepared)	✗ What NOT to Say (Avoid Bringing Up)
Driver Tracking	"Uber H3 hexagonal indexing in Redis Sets with WebSockets telemetry throttling."	Do NOT say " <i>Kalman Filter</i> ", " <i>Ramer-Douglas-Peucker</i> ", or " <i>PostGIS</i> ".
Maps & Pricing	"Centralized backend Directions proxy with coordinate-rounding Redis cache and offline TypeScript pricing engine."	Do NOT say client apps called Google Maps directly for every fare.
Financial Ledger	"Immutable double-entry wallet ledger using Prisma transactions with idempotency keys."	Do NOT say " <i>Decimal precision module</i> " (just say Prisma/PostgreSQL Decimal).
Driver Payouts	"Automated Cashfree Payouts API integration (IMPS/UPI) with webhook signature verification."	Do NOT say " <i>Dynamic RSA-OAEP 2FA</i> ".
Frontend Architecture	"3 Next.js 16 portals (App Router, SSR), React 19, Zustand for client state, TanStack Query, and Tailwind CSS."	Do NOT say you used Redux for everything; highlight Zustand for booking state.
Testing & DevOps	"Postman collection for automated endpoint testing, Docker & Docker Compose for multi-service containerization."	Do NOT say " <i>Jest unit tests across all services</i> " or " <i>NGINX reverse proxy</i> ".

💡 **Final Tip:** Speak slowly, clearly, and structure your answers with **Problem** → **Architecture/Solution** → **Result**. You have a battle-tested architecture that solves real engineering challenges!